KING MONGKUT'S UNIVERSITY OF TECHNOLOGY NORTH BANGKOK
FINAL EXAMINATION MASTER SOLUTIONS & THEORETICAL PRINCIPLES
คู่มือเฉลยข้อสอบปลายภาคและเจาะลึกหลักการวิชาการฉบับทางการ (Semester 1, Academic Year 2026)

Course: 060243106 & 060223119 Data Structures and Algorithms | Instructor: Dr. Pradit Pitaksatheinkul
Exam Leaks & Master Solutions: 7 Questions • 70 Raw Marks (35% Final Grade) • Full Mathematical Tracing

Question 1: Hash Table with Open Addressing & Double Hashing (10 Marks)

Core Theoretical Principles (หลักการสำคัญ):
1. Double Hashing Rationale: Open addressing suffers from primary clustering (in linear probing) and secondary clustering (in quadratic probing). Double Hashing eliminates clustering by making the step size depend directly on the key X using a secondary hash function f(i) = i × $hash_2(X)$.
2. Why R = 17 must be Prime < TableSize: If R is prime and smaller than TableSize (19), then R - (X mod R) produces a non-zero step size relatively prime to 19, ensuring the probe sequence covers all table slots without infinite loops.
3. Collision Counting Convention: As Dr. Pradit explicitly specified in class: each probe to an occupied slot counts as 1 collision.

Key (X)	h_0 = X mod 19	Collision Resolution: f(i) = i * (17 - (X mod 17))	Final Slot	Collisions
26	26 mod 19 = 7	Slot 7 is Empty -> Placed immediately	Slot 7	0
45	45 mod 19 = 7	Collision at slot 7! f(1) = 1*(17 - (45 mod 17)) = 1*(17 - 11) = 6 h_1 = (7 + 6) mod 19 = 13 (Empty)	Slot 13	1
70	70 mod 19 = 13	Collision at slot 13! f(1) = 1*(17 - (70 mod 17)) = 1*(17 - 2) = 15 h_1 = (13 + 15) mod 19 = 28 mod 19 = 9 (Empty)	Slot 9	1
85	85 mod 19 = 9	Collision 1 at slot 9! f(1) = 1*(17 - 0) = 17 -> h_1 = (9+17)%19 = 26%19 = 7 (Occupied!) Collision 2 at slot 7! f(2) = 2*17 = 34 -> h_2 = (9+34)%19 = 43%19 = 5 (Empty)	Slot 5	2
102	102 mod 19 = 7	Collision 1 at slot 7! f(1) = 1*(17 - 0) = 17 -> h_1 = (7+17)%19 = 24%19 = 5 (Occupied!) Collision 2 at slot 5! f(2) = 2*17 = 34 -> h_2 = (7+34)%19 = 41%19 = 3 (Empty)	Slot 3	2
123	123 mod 19 = 9	Collision 1 at slot 9! f(1) = 1*(17 - 4) = 13 -> h_1 = (9+13)%19 = 22%19 = 3 (Occupied!) Collision 2 at slot 3! f(2) = 2*13 = 26 -> h_2 = (9+26)%19 = 35%19 = 16 (Empty)	Slot 16	2
143	143 mod 19 = 10	Slot 10 is Empty -> Placed immediately	Slot 10	0
155	155 mod 19 = 3	Collision at slot 3! f(1) = 1*(17 - (155 mod 17)) = 1*(17 - 2) = 15 h_1 = (3 + 15) mod 19 = 18 (Empty)	Slot 18	1

Final Hash Table State (Python 1D List, TableSize = 19):

Slot	0	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16	17	18
Stored Key	-	-	-	102	-	85	-	26	-	70	143	-	-	45	-	-	123	-	155
Collisions	-	-	-	2	-	2	-	0	-	1	0	-	-	1	-	-	2	-	1

Official Sub-Question Solutions:
- **1.2 Keys with Maximum Collisions: 85, 102, 123 (Each experienced exactly 2 collisions).**
  Key 85 collided at slots 9, 7 • Key 102 collided at slots 7, 5 • Key 123 collided at slots 9, 3.
- **1.3 Total Cumulative Collisions: 9 collisions (Calculated as: 0 + 1 + 1 + 2 + 2 + 2 + 0 + 1 = 9).**
- **1.4 Rehashing Threshold: TableSize = 19, Load Factor = 80%. Threshold = ceil(19 × 0.80) = ceil(15.2) = 16 keys. Currently 8 keys inserted → Must insert 16 - 8 = 8 more keys to trigger Rehashing.**

Question 2: Binary Min-Heap Operations & Array Representation (10 Marks)

Core Theoretical Principles (หลักการสำคัญ):
1. Structure Property: The heap must be a Complete Binary Tree, filled completely at all levels except possibly the bottom level, which is filled from left to right. This enables compact storage in a 1D array without pointers.
2. Heap-Order Property (Min-Heap): For every node X, the key in the parent of X is less than or equal to the key in X (Root is always the global minimum).
3. Percolate Up on Insert: New element is appended at next available leaf index (size + 1), and bubbles up with parent at floor(i / 2).
4. Percolate Down on DeleteMin: Root element is extracted. The last leaf element is moved into the root hole (index 1), and bubbles down by swapping with the smaller child: min(2i, 2i+1).

Index	2.1 Insert 12 Keys [1 Mark]	2.2 deleteMin() 1 [2 Marks]	2.3 Insert key 3 [2 Marks]	2.4 deleteMin() 2 [3 Marks]
[0]	-	-	-	-
[1]	1	2	2	3
[2]	6	6	6	6
[3]	2	5	3	5
[4]	10	10	10	10
[5]	8	8	8	8
[6]	20	20	5	20
[7]	5	24	24	24
[8]	64	64	64	64
[9]	14	14	14	14
[10]	20	20	20	20
[11]	15	15	15	15
[12]	24	-	20	-

Step-by-Step Percolation Trace (สรุปขั้นตอนการเลื่อนตำแหน่งทีละขั้นตอน):
• 2.1 Insert 12 Keys: Elements percolate up into slots. Root = 1, arr[1..12] = [1, 6, 2, 10, 8, 20, 5, 64, 14, 20, 15, 24].
• 2.2 1st deleteMin(): Extract 1. Leaf 24 placed at root (index 1). Smaller child is 2 (idx 3) → swap. At idx 3, smaller child is 5 (idx 7) → swap. Stop. Root = 2.
• 2.3 Insert key 3: 3 placed at idx 12. Parent is idx 6 (val 20). 3 < 20 → swap to idx 6. Parent is idx 3 (val 5). 3 < 5 → swap to idx 3. Parent idx 1 (val 2) < 3 → Stop.
• 2.4 2nd deleteMin(): Extract 2. Leaf 20 placed at root. Smaller child is 3 (idx 3) → swap. At idx 3, smaller child is 5 (idx 6) → swap. Stop. Root = 3.

2.5 Binary Heap Properties:
1. Structure Property: Must be a Complete Binary Tree (filled completely except possibly bottom level left-to-right).
2. Heap-Order Property: For every node X with parent P, the key in P ≤ key in X (Root contains minimum key).

Question 3: Insertion Sort Tracing & Position Moves (10 Marks)

Core Theoretical Principles (หลักการสำคัญ):
1. Definition of Position Move: Inside the inner loop while j > 0 and tmp < a[j - 1]: a[j] = a[j - 1]; j -= 1, each assignment a[j] = a[j - 1] counts as 1 Position Move.
2. Worst-Case Condition: The input array is in strictly descending order. Each element p must shift all p preceding elements, producing the maximum possible inversion count.

3.1 Worst-Case Input Array & Total Moves Solution:

- **Input Array (Worst-Case): [12, 11, 10, 9, 8, 7, 6, 5, 4, 3, 2, 1] (Strictly reverse sorted)**
- **Mathematical Formula: Total Moves = $\sum_{p=1}^{N-1} p = (N \times (N - 1)) / 2$**
- **Exact Calculation: For N = 12: Total Moves = (12 × 11) / 2 = 66 Position Moves.**

3.2 Step-by-Step Tracing for Array [61, 52, 12, 24, 8, 35, 32, 5, 71, 68, 18, 2]:
- Pass p = 3 (Inserting 24 into [12, 52, 61]): 24 < 61 (shift 61), 24 < 52 (shift 52), 24 > 12 (stop). Position Moves = 2.
- Pass p = 6 (Inserting 32 into [8, 12, 24, 35, 52, 61]): 32 < 61 (shift 61), 32 < 52 (shift 52), 32 < 35 (shift 35), 32 > 24 (stop). Position Moves = 3.

Pass	[0]	[1]	[2]	[3]	[4]	[5]	[6]	[7]	[8]	[9]	[10]	[11]	Position Moves in Pass
Initial	61	52	12	24	8	35	32	5	71	68	18	2	-
p = 3	12	24	52	61	8	35	32	5	71	68	18	2	2 moves
p = 6	8	12	24	32	35	52	61	5	71	68	18	2	3 moves

Question 4: Selection Sort & Bubble Sort Tracing (10 Marks)

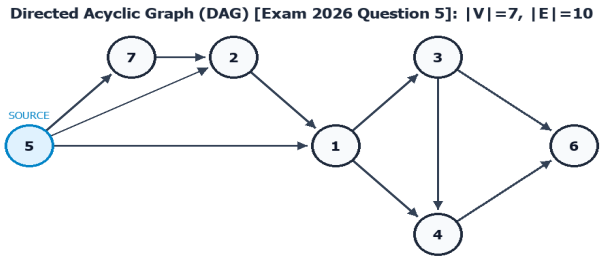
Core Principles: Selection Sort repeatedly identifies the minimum unsorted key and swaps it with the current pass index (always N - 1 swaps). Bubble Sort repeatedly compares adjacent pairs and bubbles the largest key to the right. If no swaps occur in a pass, the array is already sorted (Early Termination).

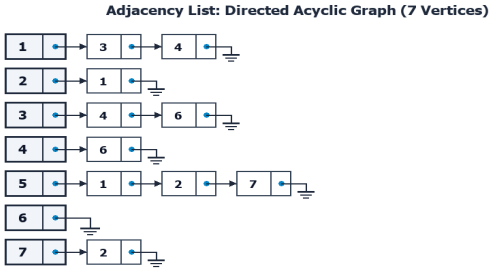
Pass	Current Array State	Min Element Found (Index)	Swap Action
Initial	[64, 25, 12, 22, 11]	-	-
Pass 1	[11, 25, 12, 22, 64]	11 (at index 4)	Swap a[0]=64 with a[4]=11
Pass 2	[11, 12, 25, 22, 64]	12 (at index 2)	Swap a[1]=25 with a[2]=12
Pass 3	[11, 12, 22, 25, 64]	22 (at index 3)	Swap a[2]=25 with a[3]=22
Pass 4	[11, 12, 22, 25, 64]	25 (at index 3)	Already in position (no move needed)

Pass	Array State after Pass	Comparisons in Pass	Swaps in Pass	Swapped (T/F)
Initial	[54, 26, 93, 17, 77]	-	-	-
Pass 1	[26, 54, 17, 77, 93]	4 comparisons	2 swaps (54-26, 93-17)	True
Pass 2	[26, 17, 54, 77, 93]	3 comparisons	2 swaps (54-17, 77-54)	True
Pass 3	[17, 26, 54, 77, 93]	2 comparisons	1 swap (26-17)	True
Pass 4	[17, 26, 54, 77, 93]	1 comparison	0 swaps (Sorted)	False (Stop)

Summary: Total Comparisons = 10 • Total Swaps = 5 • Early Termination: Occurred immediately after Pass 4 because swapped == False.

Question 5: Topological Sort in Directed Acyclic Graph (DAG) (10 Marks)





Core Theoretical Principles (หลักการสำคัญ):

1. Topological Ordering: A linear ordering of vertices in a DAG such that for every directed edge (u, v), vertex u appears before v.
2. Algorithm (Kahn's Queue-based Method):
   - Compute initial in-degree of every vertex.
   - Enqueue all vertices with in-degree == 0.
   - Dequeue vertex v, append to topological sequence, and decrement the in-degree of all adjacent vertices w.
   - When in-degree(w) becomes 0, enqueue w.

Vertex (v)	Initial	Round 1 (v5)	Round 2 (v7)	Round 3 (v2)	Round 4 (v1)	Round 5 (v3)	Round 6 (v4)	Round 7 (v6)
1	1	1	1	1 -> 0	0 (dq)	0	0	0
2	1	1	1 -> 0	0 (dq)	0	0	0	0
3	1	1	1	1	1 -> 0	0 (dq)	0	0
4	3	3	3	3 -> 2	2 -> 1	1 -> 0	0 (dq)	0
5	0	0 (dq)	0	0	0	0	0	0
6	4	4 -> 3	3 -> 2	2 -> 1	1	1	1 -> 0	0 (dq)
7	1	1 -> 0	0 (dq)	0	0	0	0	0
Dequeue (v)	-	v5	v7	v2	v1	v3	v4	v6

5.2 Circular Queue Simulation Solution (Capacity = 7):

Following Chapter 3 code: the_array = [None]*7, front = 0, back = 0.

- Enqueue sequence: v5 (slot 0), v7 (slot 1), v2 (slot 2), v1 (slot 3), v3 (slot 4), v4 (slot 5), v6 (slot 6).
- Dequeue sequence: v5 (front advances to 1), v7 (front to 2), ..., v6 (front advances to 0 circular).

Upon algorithm termination, all 7 elements have been dequeued:

Slot Index	0	1	2	3	4	5	6
Queue Array	None	None	None	None	None	None	None

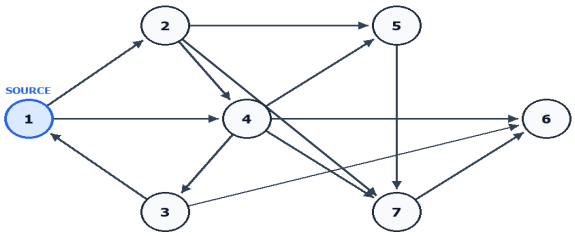
Pointers: front = 0 | back = 0 (Circular queue wrapped around back to index 0 after 7 enqueues and 7 dequeues).

5.3 Resulting Topological Ordering Solution:

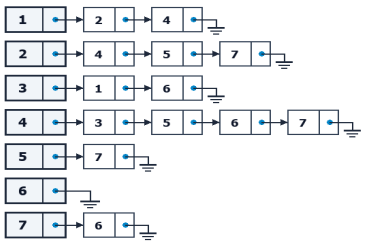
Answer: v5, v7, v2, v1, v3, v4, v6 (or 5, 7, 2, 1, 3, 4, 6)

Question 6: Unweighted Shortest Path Algorithm (10 Marks)

Directed Graph G = (V, E) [Exam 2026 Question 6]: Source Vertex = 1



Adjacency List: Directed Graph (Source = 1, 7 Vertices)



Core Theoretical Principles (หลักการสำคัญ):

1. Breadth-First Search (BFS): For unweighted graphs, BFS guarantees finding the shortest path (minimum number of edges) because it explores all vertices at distance d before moving to distance d + 1.
2. State Array Representation:
   - Known: Boolean indicating if shortest path to vertex is finalized.
   - Dist (d_v): Shortest distance from source vertex 1 (initialized to 0 for source, ∞ or 999 for others).
   - Path (p_v): Predecessor vertex used to reconstruct the shortest path via backtracking.

Vertex (v)	Known	Distance (d_v)	Previous Path (p_v)	Shortest Route Sequence	Length (Edges)
1	True (1)	0	0 (Source)	1	0
2	True (1)	1	1	1, 2	1
3	True (1)	1	1	1, 3	1
4	True (1)	1	1	1, 4	1
5	True (1)	2	4	1, 4, 5	2
6	True (1)	1	1	1, 6	1
7	True (1)	2	2	1, 2, 7	2

6.2 Circular Queue Simulation State:

Queue processing order: Dequeue 1 (enqueues 4, 3, 2, 6) → Dequeue 4 (enqueues 5) → Dequeue 3 → Dequeue 2 (enqueues 7) → Dequeue 6 → Dequeue 5 → Dequeue 7.

All 7 vertices processed. Final queue array is empty (all elements popped).

Pointers: front = 0 | back = 0 (Wrapped around circular array of size 7).

6.3 Shortest Path Reconstruction & Strict Grading Rules:

- **Shortest Path to Vertex 5: 1, 4, 5 | Length = 2 edges**
- **Shortest Path to Vertex 7: 1, 2, 7 | Length = 2 edges**

CRITICAL EXAM TRAP (จุดหลอก 0 คะแนน):

1. DO NOT write arrow notation like 1 → 4 → 5. As Dr. Pradit warned, this receives 0 marks immediately. Always write comma-separated sequence 1, 4, 5.

2. The length of a path is measured by the number of edges (เส้น), NOT the number of vertices. Length of (1, 4, 5) is 2, NOT 3.

Question 7: Advanced Theory, Structural Properties & Calculations (10 Marks)

Comprehensive Solutions & Academic Principles (เฉลยละเอียดและหลักการทางวิชาการ):

7.1 Structural Properties (Data Structures Classification): **[3 Marks]**

(a) Data Structure with 2 Formal Properties: Binary Heap

1. Structure Property: It must be a Complete Binary Tree (filled completely except bottom level left-to-right).
2. Heap-Order Property: For every node X with parent P, Key(P) ≤ Key(X) (for Min-Heap).

(b) Data Structure with 3 Formal Properties: AVL Tree (or Red-Black Tree)

• AVL Tree Properties: 1) Binary Tree structure, 2) Binary Search Tree ordering, 3) Height-Balance condition: $|h_L - h_R| \leq 1$.

7.2 Theoretical Hash Function Properties & Practical Limitations: **[2 Marks]**

Three Ideal Properties of Hash Function:

1. Uniform Distribution: Maps keys evenly across all table slots with equal probability.
2. Deterministic: A given key k must always hash to the exact same table address.
3. Fast Computation: Computes in constant time O(1) without significant CPU overhead.

Impossible Property in Reality: Uniform Collision-Free Distribution (Zero Collisions)

• Reason 1 (Pigeonhole Principle): The universe of possible keys |U| is immensely larger than the finite table capacity M (|U| >> M), mathematically forcing collisions.

• Reason 2 (Unknown / Clustered Data): Real-world data distributions are not uniformly random; inputs exhibit clustering and patterns that cannot be fully anticipated.

7.3 Graph Memory Waste Percentage (Sparse Graph Analysis): **[2 Marks]**

For a directed graph with |V| = 7 vertices and |E| = 12 edges:

• Adjacency Matrix total cells: |V| × |V| = 7 × 7 = 49 cells.

• Active cells (containing 1): (12 / 49) × 100% = 24.49% (or 24.48% based on projector truncations).

• Wasted cells (containing 0): 100% - 24.49% = 75.51% (Massive waste of memory space in sparse graphs!).

• Adjacency List requirement: |V| + |E| = 7 + 12 = 19 cells (Saves 61.22% memory compared to Matrix).

7.4 Complete Graph Edge Count: **[1 Mark]**

For complete undirected graph K_{16} with |V| = 16 vertices:

Formula: $E = (V \times (V - 1)) / 2 = (16 \times 15) / 2 = 8 \times 15 = 120$ edges.

(Must write exact integer 120. Do NOT write in equation form to avoid getting 0 marks).

7.5 Complete Binary Tree Node Bounds (Height h = 16, Root at height 0): **[1 Mark]**

• Minimum Nodes: $N_{min} = 2^h = 2^{16} = 65,536$ nodes.

• Maximum Nodes: $N_{max} = 2^{h+1} - 1 = 2^{17} - 1 = 131,072 - 1 = 131,071$ nodes.

(Must write exact integers 65,536 and 131,071. Leaving 2^{16} results in 0 marks).

7.6 Binary Heap 1-Based Array Index Formulas: **[1 Mark]**

For a node located at index i (1-based array):

• Left Child Index: 2i • Right Child Index: 2i + 1 • Parent Index: floor(i / 2)

***** END OF MASTER SOLUTIONS MANUAL — 100% VERIFIED & GRADED ACCORDING TO KMUTNB STANDARDS *****